

BCA-301 — OBJECT ORIENTED PROGRAMMING USING C++

Code: 303301

Semester	III (BCA)	Time	3 Hours
Full Marks	60	Attempt	5 of 7 questions
Compulsory	Q1 and Q2	Pattern	2 / 4 / 12 marks

CONTENTS

Unit 1 — Paradigms & Basics	 page 2
Unit 2 — Operator Overloading	 page 10
Unit 3 — Inheritance	 page 15
Unit 4 — Advanced Topics	 page 20
Most Repeated Questions	 page 29
Top 20 VVI Questions	 page 30
Important C++ Programs	 page 34
Last Minute Cheat Sheet	 page 40

Each unit has 3 parts: **Column 1** MCQ + One-Liner, **Column 2** 4-Mark Short Answers, **Column 3** 12-Mark Long Answers. Built only from the uploaded syllabus and PYQ papers (2014, 2017, 2018, 2019, 2020, 2021, 2022, 2024). Other years: Not verified from uploaded material. No question is guaranteed.

UNIT 1

Introduction to Programming Paradigms + Basics of C++

TOPICS COVERED

- Programming paradigms (procedural vs object-oriented)
- Object, class, data hiding, encapsulation, abstraction
- Structure of a C++ program, `cin` / `cout`
- Member functions (inside / outside class), scope resolution operator
- Access specifiers: `private`, `protected`, `public`
- Constructors, destructors, copy constructor, default arguments
- Inline functions, static members, `this` pointer, friend function
- Objects as function arguments, call by value / reference
- Dynamic objects with `new` and `delete`

COLUMN 1 | MCQ + ONE-LINER

One-Liner Definitions

- **Class:** user-defined data type (blueprint) that groups data and functions.
- **Object:** an instance (variable) of a class.
- **Data hiding:** keeping data **private** so outsiders cannot touch it directly.
- **Encapsulation:** **wrapping data and functions together** in one unit (class).
- **Abstraction:** showing only essential features and **hiding complexity**.
- **Message passing:** objects talk to each other by calling member functions.
- **Constructor:** special function with the **same name as class**, called automatically when an object is created.
- **Destructor:** special function with `~` before class name, called automatically when an object is destroyed.
- **Inline function:** body is substituted at the call place to save call overhead.
- **Friend function:** non-member function that can access private and protected members.
- **Static member:** one copy shared by all objects of the class.
- **Container class:** class that holds a collection of objects (example: vector).
- **Proxy class:** class that gives a limited interface to hide the real class.
- **Garbage collection:** automatic memory cleanup; **C++ does not have it**, programmer uses `delete`.

MCQs with Answers + One-liner Explanation

Q1. Which access specifier is used in a class definition by default? PYQ 2014 PYQ 2019

- (A) Protected (B) Public
(C) **Private ✓** (D) Friend

Correct Answer: (C) — Class members are private unless you write public or protected.

Q2. Which of the following is NOT a member of a class? PYQ 2014

- (A) Static function (B) **Friend function ✓**
(C) Const function (D) Virtual function

Correct Answer: (B) — A friend function is declared inside the class but it is an outside (non-member) function.

Q3. Which symbol is used to define a member of a class externally? PYQ 2019

- (A) : (B) :: ✓
(C) # (D) None of the above

Correct Answer: (B) — :: is the scope resolution operator: `void A::show() { }`.

Q4. A function that is called automatically each time an object is destroyed is a: PYQ 2018

- (A) Constructor (B) Destructor ✓
(C) Destroyer (D) Terminator

Correct Answer: (B) — Destructor (`~ClassName`) runs automatically at object destruction.

Q5. Hiding the complexity is known as: PYQ 2018

- (A) Abstraction ✓ (B) Encapsulation
(C) Data hiding (D) Composition

Correct Answer: (A) — Abstraction shows only what is needed and hides the inside details.

Q6. How many types of constructor are there in C++? PYQ 2019

- (A) 1 (B) 2
(C) 3 ✓ (D) 4

Correct Answer: (C) — Default, parameterized and copy constructor.

Q7. cout stands for: PYQ 2018

- (A) class output (B) character output
(C) console output ✓ (D) call output

Correct Answer: (C) — `cout` is the console output stream object.

Q8. A class has only a parameterized constructor. We create an object with no arguments. What happens? PYQ 2014

- (A) Runs normally (B) Compile-time error ✓
(C) Run-time error (D) Garbage value is taken

Correct Answer: (B) — The compiler does not make a default constructor when you write any constructor, so the zero-argument call fails.

Q9. Unsigned integers in C++ are: PYQ 2019

- (A) Negative numbers (B) Non-negative numbers ✓
(C) Float numbers (D) None of the above

Correct Answer: (B) — `unsigned` stores only zero and positive values.

Q10. Which of the following correctly declares an array? PYQ 2019

- (A) `int array[10];` ✓ (B) `int array;`
(C) `array{10};` (D) `array array[10];`

Correct Answer: (A) — Syntax is `datatype name[size];`.

Q11. C++ was initially known as: PYQ 2025 (implied from study material)

- (A) C++ (B) C with objects
(C) C with classes ✓ (D) Object C

Correct Answer: (C) — Bjarne Stroustrup originally called it 'C with classes' before renaming to C++ in 1984.

Q12. Memory for data members is allocated when: PYQ 2025 (class and objects study material)

- (A) A class is declared (B) An object is created ✓
(C) A function is called (D) A header is included

Correct Answer: (B) — A class is just a blueprint; memory is allocated only when an object is created.

Q13. Which access modifier makes members accessible everywhere (inside class, derived class, outside)? PYQ 2025

- (A) private (B) protected
(C) public ✓ (D) friend

Correct Answer: (C) — public members can be accessed from anywhere in the program.

Q14. Which access modifier allows access inside the same class AND derived classes, but NOT outside? PYQ 2025

- (A) public (B) private
(C) protected ✓ (D) static

Correct Answer: (C) — protected = same class + derived classes, not outside. private = same class only.

Q15. What is the smallest meaningful unit of a C++ program? PYQ 2025

- (A) Function (B) Statement
(C) Token ✓ (D) Variable

Correct Answer: (C) — A token is the smallest unit: keywords, identifiers, constants, operators, punctuators.

Q16. A proxy class is used to: PYQ 2025

- (A) Overload operators (B) Provide controlled access to another class ✓
(C) Create virtual functions (D) Handle exceptions

Correct Answer: (B) — A proxy class acts as a stand-in or interface that controls access to the real class.

Q17. What is an ambiguity in C++? PYQ 2025

- (A) A syntax error (B) A run-time crash
(C) When the compiler cannot decide which function to call ✓ (D) A missing semicolon

Correct Answer: (C) — Ambiguity arises mainly in multiple inheritance when two base classes have the same function name.

COLUMN 2 | 4-MARK SHORT ANSWERS

Q1. What is the scope resolution operator? Give example. PYQ 2017

Ans.

The **scope resolution operator (::)** links a member with its class, or accesses a global variable hidden by a local one. It is used to define a member function outside the class.

```
class A { public: void show(); };
void A::show() { cout << "Hello"; }
```

Q2. Explain inline functions. PYQ 2017 PYQ 2020 PYQ 2021

Ans.

An **inline function** is declared with the keyword `inline`. The compiler replaces the function call with the function body, so **function-call overhead is saved** and speed improves. Use it only for small functions.

```
inline int square(int x) { return x * x; }
```

Q3. What is a static data member? Give its benefits. PYQ 2019 PYQ 2020

Ans.

A **static data member** is declared with `static`. Only **one copy** exists for all objects, and it is created before any object. It is defined outside the class: `int A::count = 0;`. Benefit: useful for **counting objects** and sharing common data.

Q4. What is the this pointer? PYQ 2022

Ans.

`this` is an **implicit pointer** available in every non-static member function. It holds the **address of the object that called the function**. Use: to return the current object (`return *this;`) or to separate member and parameter names.

Q5. Write a sample code to show the difference between C and C++. PYQ 2025

Ans.

C is a **procedural** language (functions only). C++ is **object-oriented** (classes, objects, overloading, inheritance). Key difference: C++ groups data and functions into a class.

```
// --- C style ---
#include <stdio.h>
void greet() { printf("Hello, C!\n"); }
int main() { greet(); return 0; }

// --- C++ style ---
#include <iostream>
using namespace std;
class Greeter {
public:
    void greet() { cout << "Hello, C++!" << endl; }
};
int main() { Greeter g; g.greet(); return 0; }
```

Q6. Explain token in C++. PYQ 2025

Ans.

A **token** is the **smallest unit of a C++ program** that is meaningful to the compiler. The compiler splits the source code into tokens before processing.

- **Keywords:** reserved words like `int`, `class`, `for`, `return`, `virtual`.
- **Identifiers:** user-defined names like `roll`, `Student`, `show`.
- **Constants / Literals:** `10`, `3.14`, `'A'`, `true`, `"Hello"`.
- **Operators:** `+`, `-`, `*`, `<<`, `::`, `->`, `=`.

- **Punctuators / Separators:** {, }, ;, ,, (,).

Q7. Define proxy classes. PYQ 2025

Ans.

A **proxy class** is a class that acts as a **stand-in or interface** to another (real) class. It provides **limited or controlled access** to the real class and hides its implementation details.

- Also called a wrapper, surrogate, or representative class.
- Use case: lazy initialisation — load the heavy object only when actually needed.
- C++ **smart pointers** (`unique_ptr`, `shared_ptr`) are real-life proxy classes for raw pointers.
- Advantage: hides complexity, controls access, saves memory until needed.

```
class Image { public: void show() { cout << "Showing image\n"; } };
class ImageProxy {
    Image* real;
public:
    ImageProxy() { real = nullptr; }
    void show() {
        if (!real) real = new Image();    // lazy load
        real->show();
    }
};
```

Q8. What is ambiguity in C++? How is it resolved? PYQ 2025

Ans.

Ambiguity in C++ occurs when the compiler **cannot decide which function or member to call** because two or more candidates match equally.

- **Multiple inheritance:** two base classes have the same function name `show()`. Calling `obj.show()` is ambiguous.
- **Resolution 1:** Use scope resolution operator: `obj.A::show();`
- **Resolution 2:** Override the function in the derived class.
- **Resolution 3:** Use a **virtual base class** to avoid duplicate copies.
- **Function overloading ambiguity:** when implicit type conversion makes two overloaded functions equally good. Fix by using explicit casting.

COLUMN 3 | 12-MARK LONG ANSWERS

Q1. Explain the object-oriented paradigm with its features. Differentiate between OOP and POP.

PYQ 2017 PYQ 2019 PYQ 2020 PYQ 2021 PYQ 2022 PYQ 2024

INTRODUCTION

Programming paradigm means a style of writing programs. C++ supports both procedural and object-oriented style.

DEFINITION

OOP is a method of programming in which a program is built from **objects** that contain both data and functions.

POINTS: FEATURES OF OOP

- **Class and Object:** class is blueprint, object is instance.
- **Encapsulation:** data and functions bound together.
- **Data hiding:** private data is protected from outside use.
- **Abstraction:** only necessary details are shown.
- **Inheritance:** new class acquires properties of an old class (reusability).
- **Polymorphism:** one name, many forms (overloading, virtual functions).
- **Dynamic binding:** function to run is decided at run time.
- **Message passing:** objects communicate through function calls.

Point	POP	OOP
Focus	Functions (procedures)	Objects (data)
Approach	Top-down	Bottom-up
Data security	Data is global, less safe	Data hiding gives high security
Reusability	Less	High (inheritance)
Overloading	Not possible	Possible
Examples	C, Pascal	C++, Java

EXAMPLE

A **Student** class holds roll number (data) and **show()** (function) together. Many student objects are made from it.

CONCLUSION

OOP is better for large programs because of security, reusability and easy maintenance.

Q2. What is a constructor? Explain its types and characteristics. Write a program for the copy constructor.

PYQ 2014 PYQ 2017 PYQ 2018 PYQ 2019 PYQ 2020 PYQ 2021 PYQ 2022 PYQ 2024

INTRODUCTION

When an object is created it needs initial values. A constructor does this job automatically.

DEFINITION

A **constructor** is a special member function with the **same name as the class** that initialises an object.

POINTS: CHARACTERISTICS AND TYPES

- Same name as class, **no return type** (not even void).
- Called **automatically** at object creation; normally in the **public** section.
- Can be **overloaded** and can have **default arguments**; cannot be virtual or inherited.
- **Default constructor:** no arguments. `Student() { roll = 0; }`
- **Parameterized constructor:** takes arguments. `Student(int r) { roll = r; }`
- **Copy constructor:** makes an object from another object. `Student(const Student &s) { roll = s.roll; }`
- **Destructor** `~Student()`: no arguments, no return type, one per class. If not written, the **compiler creates an empty destructor**.
- Copy constructor is called when an object is initialised from another, passed by value or returned by value. Write your own when the class has **pointers** (deep copy).

- If only a parameterized constructor exists and an object is made with no arguments, it is a **compile-time error**.

PROGRAM

Constructor, copy constructor and destructor together:

```
#include <iostream>
using namespace std;
class Distance {
    int feet, inches;
public:
    Distance(int f = 0, int i = 0) {           // parameterized + default values
        feet = f; inches = i;
        cout << "Constructor called\n";
    }
    Distance(const Distance &d) {             // copy constructor
        feet = d.feet; inches = d.inches;
        cout << "Copy constructor called\n";
    }
    void show() { cout << feet << " ft " << inches << " in\n"; }
    ~Distance() { cout << "Destructor called\n"; }
};
int main() {
    Distance d1(5, 8);
    Distance d2 = d1;
    d1.show();
    d2.show();
    return 0;
}
```

CONCLUSION

Constructors and destructors manage object creation and cleanup safely.

Q3. What is a friend function? Why is it required? Write a program.

PYQ 2014 PYQ 2018 PYQ 2020 PYQ 2021 PYQ 2022 PYQ 2024

INTRODUCTION

Private members cannot be used outside the class, but sometimes an outside function needs them.

DEFINITION

A **friend function** is a **non-member function** declared inside the class with the keyword **friend**, which can access private and protected members.

POINTS

- Declared inside the class, defined outside **without ::** and **without friend**.
- **Not called with object and dot**; objects are passed as arguments.
- Can be a friend of more than one class (useful to work on two classes).
- Needed for overloading << and >>.
- Demerit: breaks data hiding, so use carefully.

PROGRAM

Friend function printing a private member:

```
#include <iostream>
using namespace std;
class Box {
    int width;
public:
    Box(int w) { width = w; }
    friend void printWidth(Box b);
};
void printWidth(Box b) { cout << "Width = " << b.width; }
int main() { Box b(10); printWidth(b); return 0; }
// Output: Width = 10
```

CONCLUSION

Friend functions give controlled outside access to private data.

Q4. What is the significance of static data members and static member functions in C++? PYQ 2025**INTRODUCTION**

Normally each object of a class has its own separate copy of every data member. But sometimes we need data that is shared by ALL objects. Static members solve this.

DEFINITION

A **static data member** is a variable shared by all objects of the class (only ONE copy exists). A **static member function** is a function that can be called **without creating an object** and can only access static members.

POINTS: STATIC DATA MEMBER

- Declared with `static` inside the class and **defined outside**: `int A::count = 0;`
- **Only ONE copy** regardless of how many objects are created.
- Exists before any object is created and destroyed after all objects are destroyed.
- Used for: counting objects, sharing common configuration, singleton pattern.
- **Static member function**: declared as `static void show()`, called as `ClassName::show()`.
- Static function has **NO this pointer** — it does not belong to any single object.
- A static function can **only access static data members**, not normal (instance) members.
- Significance: saves memory, provides class-level (shared) state, accessible without an object.

PROGRAM

Counting student objects automatically using static member:

```
#include <iostream>
using namespace std;
class Student {
    int roll;
    static int count;          // one shared copy
public:
    Student(int r) { roll = r; count++; }
    ~Student() { count--; }
    static void showCount() { // no object needed
        cout << "Total Students: " << count << endl;
    }
};
int Student::count = 0;       // definition outside class
int main() {
    Student::showCount();     // 0
    Student s1(1), s2(2);
    Student::showCount();     // 2
    {
        Student s3(3);
        Student::showCount(); // 3
    }                          // s3 destroyed here
    Student::showCount();     // 2
    return 0;
}
```

CONCLUSION

Static members are essential for managing class-level shared state without using global variables.

PYQ ALERT

Asked in: OOP vs POP (2017, 2019, 2020, 2021, 2022, 2024); Constructor / Destructor / Copy constructor (2014, 2017, 2018, 2019, 2020, 2021, 2022, 2024); Friend function (2014, 2018, 2020, 2021, 2022, 2024); Inline (2017, 2020, 2021); Static member (2019, 2020); Class vs structure (2018); new / delete (2018, 2021); this pointer (2022); Scope resolution (2017); Static data/member function significance (2025, 12-mark); C vs C++ code (2025, 4-mark); Token (2025, 2-mark); Proxy classes (2025, 2-mark); Ambiguity (2025, 2-mark); New and delete (2025, 2-mark); Access specifier (2025, 2-mark)

Frequency: High

UNIT 2

Operator Overloading

TOPICS COVERED

- Fundamentals and restrictions of operator overloading
- Operator functions as class members vs friend functions
- Overloading stream operators (<<, >>)
- Binary and unary operators (prefix / postfix)
- Converting between types (basic and class)

COLUMN 1 | MCQ + ONE-LINER

One-Liner Definitions

- **Operator overloading**: giving an existing operator an extra meaning for user-defined types.
- **Operator function**: function written with the keyword **operator** followed by the symbol.
- **Compile-time polymorphism**: operator overloading is decided at compile time.
- **Insertion operator** <<: sends data to output stream.
- **Extraction operator** >>: takes data from input stream.
- **Conversion function**: member function `operator type()` that converts class to basic type.
- Operators that **cannot be overloaded**: `::`, `.`, `.*`, `?:`, `sizeof`.

MCQs with Answers + One-liner Explanation

Q1. Which keyword is used to overload an operator? [Practice](#)

- (A) overload (B) **operator** ✓
(C) friend (D) virtual

Correct Answer: (B) — Syntax: `return_type operator + (args)`.

Q2. A binary operator overloaded as a MEMBER function takes how many arguments? [Practice](#)

- (A) Zero (B) **One** ✓
(C) Two (D) Three

Correct Answer: (B) — The left operand is the calling object, so only the right operand is passed.

Q3. A binary operator overloaded as a FRIEND function takes how many arguments? [Practice](#)

- (A) One (B) **Two** ✓
(C) Three (D) None

Correct Answer: (B) — A friend has no `this`, so both operands are passed.

Q4. Which operator can NOT be overloaded? [Practice](#)

- (A) + (B) <<
(C) **::** ✓ (D) []

Correct Answer: (C) — Scope resolution `::` (also `.`, `.*`, `?:`, `sizeof`) cannot be overloaded.

Q5. Which kind of function is normally used to overload << and >> for a class?

PYQ 2014

PYQ 2018

(A) Friend function ✓

(B) Static function

(C) Inline function

(D) Virtual function

Correct Answer: (A) — Left operand is `cout` / `cin`, not our object, so a friend (non-member) is used.

Q6. Operator overloading is a type of:

Practice

(A) Run-time polymorphism

(B) Compile-time polymorphism ✓

(C) Inheritance

(D) Abstraction

Correct Answer: (B) — The compiler decides which operator function to call.

Q7. How is the postfix ++ distinguished from prefix ++ in operator functions?

PYQ 2019

(A) By a dummy int argument ✓

(B) By a friend keyword

(C) By a return type of void

(D) By a static keyword

Correct Answer: (A) — `operator++(int)` is postfix; `operator++()` is prefix.

COLUMN 2 | 4-MARK SHORT ANSWERS

Q1. What is an operator function? Describe its syntax. PYQ 2014 PYQ 2025

Ans.

An **operator function** defines the new meaning of an operator for a class. Syntax:

```
return_type operator op (arguments) { /* body */ }
// Example
Complex operator+(Complex c);
```

Q2. Write syntax for overloading the extraction and insertion operators with a friend function.

PYQ 2014 PYQ 2018 PYQ 2025

Ans.

Both are friend functions defined outside the class. They **return the stream by reference** so chaining like `cout << a << b;` works.

```
friend ostream& operator<<(ostream &out, const ClassName &obj);
friend istream& operator>>(istream &in, ClassName &obj);
```

Q3. How many arguments are required for an overloaded binary operator? PYQ 2017

Ans.

As a **member function: one argument** (left operand is the calling object). As a **friend function: two arguments**. For a unary operator: member = **zero**, friend = **one**.

Q4. What are the restrictions on operator overloading? PYQ 2018

Ans.

- Only **existing operators** can be overloaded; new symbols cannot be created.
- `::`, `.`, `.*`, `?:`, `sizeof` **cannot** be overloaded.
- **Number of operands and precedence cannot change.**
- At least one operand must be a **user-defined type**.
- `=`, `()`, `[]`, `->` can be overloaded only as **member functions**.

COLUMN 3 | 12-MARK LONG ANSWERS

Q1. What is operator overloading? Explain with restrictions. Write a C++ program to overload + (and -, *) for complex numbers. PYQ 2014 PYQ 2017 PYQ 2018 PYQ 2019 PYQ 2022 PYQ 2025

INTRODUCTION

Normal operators work only on built-in types. Operator overloading lets them work on objects also.

DEFINITION

Operator overloading means giving an operator an **additional meaning** for a user-defined type, without changing its original meaning.

POINTS

- Done with an **operator function**: `return_type operator op (args)`.
- Member function: left operand is the calling object (`c3 = c1 + c2`; becomes `c1.operator+(c2)`).
- Friend function: used when the left operand is not our object (example: `cout << obj`).
- **Unary** operators: prefix is `operator++()`, postfix is `operator++(int)` (dummy int).
- Restrictions: `::`, `.`, `*`, `?:`, `sizeof` cannot be overloaded; operand count and precedence cannot change; at least one operand must be a class type.

PROGRAM

Complex numbers with `+`, `-`, `*`, `<<`, `>>`:

```
#include <iostream>
using namespace std;
class Complex {
    float re, im;
public:
    Complex(float r = 0, float i = 0) { re = r; im = i; }
    Complex operator+(Complex c) { return Complex(re + c.re, im + c.im); }
    Complex operator-(Complex c) { return Complex(re - c.re, im - c.im); }
    Complex operator*(Complex c) {
        return Complex(re * c.re - im * c.im, re * c.im + im * c.re);
    }
    friend ostream& operator<<(ostream &out, const Complex &c);
    friend istream& operator>>(istream &in, Complex &c);
};
ostream& operator<<(ostream &out, const Complex &c) {
    out << c.re << " + " << c.im << "i";
    return out;
}
istream& operator>>(istream &in, Complex &c) {
    in >> c.re >> c.im;
    return in;
}
int main() {
    Complex a(2, 3), b(4, 5);
    cout << "Sum = " << a + b << endl;
    cout << "Diff = " << a - b << endl;
    cout << "Prod = " << a * b << endl;
    return 0;
}
```

CONCLUSION

Operator overloading makes object code short, readable and natural. It is **compile-time polymorphism**.

Q2. Explain the conversion of basic data type to class type and class type to basic type with example. PYQ 2014 PYQ 2025

INTRODUCTION

Compiler converts basic types automatically, but for classes we must tell it how.

DEFINITION

Type conversion means changing data from one type to another. For classes it is done by a **constructor** (basic to class) and a **conversion function** (class to basic).

POINTS

- **Basic to Class:** use a **constructor with one argument**.
- **Class to Basic:** use a **conversion function** `operator type()`.
- Conversion function rules: must be a member, **no return type, no arguments**.

EXAMPLE

Meter class converting from and to float:

```
#include <iostream>
using namespace std;
class Meter {
    float m;
public:
    Meter() { m = 0; }
    Meter(float x) { m = x; }           // basic -> class
    operator float() { return m; }     // class -> basic
};
int main() {
    Meter a;
    a = 5.5;                          // float to Meter (constructor called)
    float f = a;                      // Meter to float (conversion function)
    cout << f;                        // 5.5
    return 0;
}
```

CONCLUSION

Constructors convert **to** class type; conversion functions convert **from** class type.

PYQ ALERT

Asked in: 2014 (Q2a, Q2c, Q5), 2017 (Q4), 2018 (Q2b, Q4), 2019 (Q6), 2020 (Q1f), 2021 (Q1d, Q1i), 2022 (extractor Q1f, Q7). No direct question in the uploaded 2024 paper.; Overload '+' operator program (2025, Q2e)

Frequency: High

UNIT 3

Inheritance

TOPICS COVERED

- Base and derived classes
- Constructors and destructors in derived classes
- Public, private and protected inheritance
- Types of inheritance and inheritance hierarchy
- Abstract classes
- Multiple inheritance, ambiguity and virtual base class (links with Unit 4)

COLUMN 1 | MCQ + ONE-LINER

One-Liner Definitions

- **Inheritance:** a new class (derived) **acquires properties** of an existing class (base).
- **Base class** (super / parent): the class being inherited from.
- **Derived class** (sub / child): the class that inherits.
- **Protected member:** accessible in the class and its derived classes, not outside.
- **Overriding:** derived class gives a new body to a base function with the **same signature**.
- **Abstract class:** class with at least one **pure virtual function**; its object cannot be created.
- **Virtual base class:** base inherited with **virtual** to avoid **duplicate copies** in multiple inheritance.

MCQs with Answers + One-liner Explanation

Q1. Which is a mechanism by which one object acquires the properties of another? PYQ 2018

- | | |
|--------------------------|------------------|
| (A) Encapsulation | (B) Abstraction |
| (C) Inheritance ✓ | (D) Polymorphism |

Correct Answer: (C) — Inheritance lets a derived class reuse the base class members.

Q2. What does a derived class NOT inherit from the base class? PYQ 2014

- | | |
|--------------------------------|-------------------------------|
| (A) Friends | (B) Operator = () members |
| (C) Constructor and destructor | (D) All of the above ✓ |

Correct Answer: (D) — Friends, assignment operator and constructors / destructors are not inherited.

Q3. Adding a derived class to a base class requires fundamental changes to the base class. PYQ 2018

- | | |
|----------|--------------------|
| (A) True | (B) False ✓ |
|----------|--------------------|

Correct Answer: (B) — The base class stays unchanged; that is the benefit of inheritance.

Q4. What is the purpose of an abstract class? PYQ 2019

- | | |
|--|--|
| (A) To help with database connectivity | (B) To provide data input to other classes |
| (C) To provide security to other classes | (D) To provide an appropriate base class from which other classes can inherit ✓ |

Correct Answer: (D) — An abstract class is a common base that derived classes complete.

Q5. You want common functions in a class and want to defer implementation of some functions to derived classes. You use: PYQ 2018

- (A) An interface (B) **An abstract class** ✓
(C) A friend class (D) A static class

Correct Answer: (B) — Pure virtual functions make the class abstract and force derived classes to implement them.

Q6. State True or False: A base class pointer cannot point to a derived class object. PYQ 2014

- (A) True (B) **False** ✓

Correct Answer: (B) — A base pointer CAN point to a derived object. This is the base of virtual functions.

Q7. Default inheritance mode when you write `class B : A` (no keyword) is: Practice

- (A) public (B) protected
(C) **private** ✓ (D) friend

Correct Answer: (C) — For a `class`, the default is private (for a `struct` it is public).

Q8. Which feature removes the diamond (duplicate base copy) problem? Practice

- (A) Friend function (B) **Virtual base class** ✓
(C) Static member (D) Inline function

Correct Answer: (B) — Declare the base as `virtual` in the intermediate classes.

COLUMN 2 | 4-MARK SHORT ANSWERS

Q1. Explain the relationship between base class and derived class. PYQ 2018

Ans.

It is an **"is-a" relationship**. The derived class **gets all accessible members** of the base class and can add its own members or override base functions. The base class stays unchanged. Syntax: `class Derived : public Base { };`

Q2. What are super, base, parent and inherited classes? PYQ 2020

Ans.

Super, base and parent are different names for the **original class** that gives its properties. **Derived, child, sub and inherited class** are names for the **new class** that receives them. So Base = Super = Parent and Derived = Sub = Child.

Q3. In what order are constructors and destructors called in inheritance? PYQ 2024 (constructors in inherited classes)

Ans.

Constructors: base class first, then derived class. **Destructors:** derived class first, then base class (reverse order). Arguments for a base constructor are passed with an **initialiser list**: `Derived(int a, int b) : Base(a) { }.`

Q4. What is function overriding? PYQ 2024

Ans.

When a derived class defines a function with the **same name, same arguments and same return type** as a base class function, the derived version **replaces** the base one for derived objects. With `virtual`, the correct version is chosen at run time.

Q5. Explain the concepts of abstract classes and virtual base classes with suitable examples. PYQ 2025

Ans.

Abstract class: contains at least one **pure virtual function** (`= 0`). Its object cannot be created. It forces derived classes to implement the function.

- **Pure virtual function:** `virtual void draw() = 0;`
- **Virtual base class:** base declared with `virtual` keyword in intermediate classes to avoid the **diamond problem** (duplicate copies in hybrid inheritance).
- Both improve OOP design: abstract class gives a common interface; virtual base removes ambiguity.

```
// Abstract class
class Shape { public: virtual void draw() = 0; }; // can't create Shape object
class Circle : public Shape { public: void draw() { cout << "Circle\n"; } };

// Virtual base class (diamond fix)
class A { public: int x; };
class B : virtual public A { };
class C : virtual public A { };
class D : public B, public C { }; // only ONE copy of A::x
```

COLUMN 3 | 12-MARK LONG ANSWERS

Q1. What is inheritance? Describe various types of inheritance with example.

PYQ 2014 PYQ 2017 PYQ 2018 PYQ 2021 PYQ 2024 PYQ 2025

INTRODUCTION

Writing the same code again and again wastes time. Inheritance gives **reusability**.

DEFINITION

Inheritance is the process of creating a new class (derived) from an existing class (base) so that the derived class acquires the base class properties.

POINTS: TYPES OF INHERITANCE

- **Single:** one base, one derived. `class B : public A {};`
- **Multiple:** more than one base, one derived. `class C : public A, public B {};`
- **Multilevel:** chain A to B to C. `class B : public A {};` `class C : public B {};`
- **Hierarchical:** one base, many derived. `class B : public A {};` `class C : public A {};`
- **Hybrid:** combination of two or more types (ambiguity can come; solved by virtual base class).

```
Single:      A --> B
Multiple:    A --> C <-- B
Multilevel:  A --> B --> C
Hierarchical: A --> B
              A --> C
Hybrid:      A --> B --> D
              A --> C --> D
```

EXAMPLE (MULTILEVEL)

An object of `Topper` can call `name()`, `roll()` and `rank()`.

```
class Person { public: void name() { cout << "Person\n"; } };
class Student : public Person { public: void roll() { cout << "Student\n"; } };
class Topper : public Student { public: void rank() { cout << "Topper\n"; } };
```

CONCLUSION

Inheritance gives code reuse, saves time and makes programs easy to extend.

Q2. Explain the visibility of base class members for the access specifiers private, protected and public while creating the derived class. PYQ 2018 PYQ 2022

INTRODUCTION

Access specifiers control who can use a member.

DEFINITION

private = only same class; **protected** = same class and derived classes; **public** = everywhere.

POINTS

- **Private members of base are never directly accessible** in the derived class (use public / protected functions).
- Default inheritance mode for a class is **private**.
- Public inheritance is the most common ("is-a" relation).

Base member	Public inheritance	Protected inheritance	Private inheritance
public	public	protected	private
protected	protected	protected	private
private	not accessible	not accessible	not accessible

EXAMPLE

With public inheritance, `y` is protected and `z` is public in `B`; `x` is not accessible.

```
class A { private: int x; protected: int y; public: int z; };
class B : public A { /* y protected, z public, x not accessible */ };
```

CONCLUSION

Choose the inheritance mode to control how much of the base class is exposed.

Q3. Discuss virtual base class with an example. Also explain ambiguity in multiple inheritance and how to resolve it.

PYQ 2020 PYQ 2022 PYQ 2024 (ambiguity)

INTRODUCTION

In hybrid inheritance, the final class may receive **two copies** of the same grand-base class. This is the **diamond problem** and causes ambiguity.

DEFINITION

A **virtual base class** is a base class declared with the keyword `virtual` in the derived classes so that **only one copy** reaches the final class.

POINTS: AMBIGUITY AND HOW TO RESOLVE IT

- If two base classes have a function with the same name, `obj.show()` is **ambiguous**.
- Resolve 1: use the **scope resolution operator**, `obj.A::show()`;
- Resolve 2: **override** the function in the derived class.
- Resolve 3: use a **virtual base class** for the diamond case.

EXAMPLE

Without `virtual`, `d.x` gives an ambiguity error. With `virtual`, only one `x` exists.

```
#include <iostream>
using namespace std;
class A { public: int x; };
class B : virtual public A { };
class C : virtual public A { };
class D : public B, public C { };
int main() {
    D d;
    d.x = 10;           // no ambiguity, only one copy of x
    cout << d.x;
    return 0;
}
```

CONCLUSION

Virtual base class removes duplication and ambiguity in hybrid inheritance.

PYQ ALERT

Asked in: Types of inheritance (2014, 2017, 2018, 2021, 2024); Access specifier visibility (2018, 2022); Abstract class (2017, 2018, 2019, 2022, 2024); Base / derived relationship (2018, 2020); Virtual base class (2020, 2022); Ambiguity in multiple inheritance (2024); Constructors in inherited classes (2024); Single inheritance program (2024); Abstract class + virtual base class (2025, 4-mark); Types of inheritance (2025)

Frequency: High

UNIT 4

Advanced Topics (Multiple Inheritance, Virtual Base Class, Pointers to Classes / Members, Templates, Exception Handling, File Handling)

TOPICS COVERED

- Multiple inheritance and virtual base classes (see Unit 3)
- Pointers to objects and pointers to class members
- Virtual functions, dynamic binding, virtual destructors
- Templates (function template, class template)
- Exception handling (`try`, `catch`, `throw`)
- File handling (streams, file modes, sequential and random access)

COLUMN 1 | MCQ + ONE-LINER

One-Liner Definitions

- **Pointer to object:** pointer that stores the address of an object; members are used with `->`.
- **Virtual function:** base function declared `virtual` so the **derived version runs** through a base pointer.
- **Pure virtual function:** virtual function with no body, written `= 0`.
- **Dynamic binding (late binding):** function call decided **at run time**.
- **Virtual destructor:** makes the derived destructor also run when deleting through a base pointer.
- **Template:** blueprint that lets one function or class work with **any data type**.
- **Exception:** a run-time error (example: divide by zero).
- **Stream:** flow of data between the program and a device or file.
- **seekg() / tellg():** `seekg` moves the read pointer; `tellg` gives its current position.

MCQs with Answers + One-liner Explanation

Q1. Which of the following is referred to by pointers to members? PYQ 2014

- (A) Static members of class objects **(B) Non-static members of class objects ✓**
 (C) Referring to whole class (D) None of the above

Correct Answer: (B) — A pointer to member points to a non-static member of class objects.

Q2. Which of the following is used to implement C++ interfaces? PYQ 2019

- (A) Absolute variables **(B) Abstract classes ✓**
 (C) Constant variables (D) None of the above

Correct Answer: (B) — Abstract classes with pure virtual functions act as interfaces.

Q3. Identify the correct statement about namespace. PYQ 2019

- (A) Namespace is used to group classes, objects and functions ✓** (B) Namespace marks the beginning of the program
 (C) Namespace is used to separate the class, objects (D) None of the above

Correct Answer: (A) — A namespace groups names to avoid clashes.

Q4. Which is more efficient while calling functions with big data? Practice PYQ 2019

- (A) Call by value
(B) **Call by reference** ✓
(C) Call by pointer
(D) None of the above

Correct Answer: (B) — Call by reference does not copy the data, so it is faster. (Option given in the 2019 paper.)

Q5. Which blocks are used in C++ exception handling? Practice

- (A) **try, catch, throw** ✓
(B) if, else, switch
(C) open, close, read
(D) start, stop, end

Correct Answer: (A) — try watches, throw raises, catch handles.

Q6. Which keyword is used to define a generic function or class? Practice

- (A) generic
(B) **template** ✓
(C) virtual
(D) friend

Correct Answer: (B) — `template <class T>` makes code work for any data type.

Q7. Which header file is needed for file handling? Practice

- (A) `<iostream>`
(B) **`<fstream>`** ✓
(C) `<iomanip>`
(D) `<cstring>`

Correct Answer: (B) — `<fstream>` gives `ifstream`, `ofstream` and `fstream`.

Q8. Which function moves the read pointer in a file? Practice

- (A) `tellg()`
(B) **`seekg()`** ✓
(C) `flush()`
(D) `close()`

Correct Answer: (B) — `seekg()` moves the get (read) pointer; `tellg()` only tells its position.

COLUMN 2 | 4-MARK SHORT ANSWERS

Q1. Write the syntax of a function template. PYQ 2017

Ans.

T is a placeholder type. The compiler creates the correct version when the function is called with `int`, `float`, `char` etc.

```
template <class T>
T function_name(T a, T b) { /* body */ }
```

Q2. What is a pure virtual function? PYQ 2017 PYQ 2021

Ans.

A **pure virtual function** has no definition in the base class: `virtual void area() = 0;`. A class with one or more pure virtual functions becomes an **abstract class**, and derived classes **must override** it.

Q3. Write the file opening modes. PYQ 2022 (ate vs out)

Ans.

`ios::in` (read), `ios::out` (write, erases old content), `ios::app` (append at end), `ios::ate` (open and go to end, writing allowed anywhere), `ios::trunc` (delete old content), `ios::binary` (binary mode). Combine using `|`.

```
fstream f("a.txt", ios::in | ios::out);
```

Q4. What are manipulators? Differentiate `setw()` and `setfill()`. PYQ 2021 PYQ 2022 PYQ 2024

Ans.

Manipulators are functions used with `<<` to **format output** (example: `endl`, `setw`, `setprecision`). `setw(n)` sets the **field width** for the next output. `setfill(c)` sets the **filling character** for the empty space. Header: `<iomanip>`.

Q5. What do you mean by exception handling? PYQ 2025

Ans.

Exception handling is a mechanism to detect and deal with **run-time errors** so the program does not crash. C++ uses three keywords:

- **try**: encloses code that may cause an error.
- **throw**: raises (sends) the exception when an error occurs.
- **catch**: catches and handles the exception.
- `catch(...)` catches any type. Multiple `catch` blocks can follow one `try`.
- Benefit: keeps error-handling code **separate** from normal code.

```
try {
    if (b == 0) throw "Division by zero!";
    cout << a / b;
}
catch (const char* e) { cout << e; }
```

COLUMN 3 | 12-MARK LONG ANSWERS

Q1. What is a virtual function? Explain dynamic binding and abstract class with example.

PYQ 2017 PYQ 2019 PYQ 2020 PYQ 2021 PYQ 2022 PYQ 2024

INTRODUCTION

When a base pointer points to a derived object, we want the **derived** function to run. Virtual functions do this.

DEFINITION

A **virtual function** is a member function of the base class declared with `virtual` and redefined in the derived class.

POINTS

- Steps: (1) declare `virtual` in base, (2) redefine in derived with same signature, (3) call through a **base class pointer**.
- **Dynamic binding**: call is resolved at **run time** using the real object type. Without `virtual`, **static binding** happens at compile time.
- **Abstract class**: has a pure virtual function (`= 0`); object cannot be created, pointers can.
- **Virtual destructor**: write `virtual ~Shape(){}` so delete `p`; also runs the derived destructor (no memory leak).

PROGRAM

Base pointer calling the derived function:

```
#include <iostream>
using namespace std;
class Shape {
public:
    virtual void draw() { cout << "Drawing shape\n"; }
    virtual ~Shape() { }
};
class Circle : public Shape {
public:
    void draw() { cout << "Drawing circle\n"; }
};
int main() {
    Shape *p;
    Circle c;
    p = &c;
    p->draw();    // Drawing circle (dynamic binding)
    return 0;
}
```

CONCLUSION

Virtual functions give **run-time polymorphism**.

Q2. Define exception handling. Explain the use of try, catch and throw with example.

PYQ 2014 PYQ 2017 PYQ 2018 PYQ 2020 PYQ 2022

INTRODUCTION

Programs can fail at run time (divide by zero, file not found). We should handle these safely.

DEFINITION

Exception handling is a mechanism to detect and handle run-time errors so that the program does not crash.

POINTS: KEYWORDS AND POINTS

- **try**: block that contains code that may cause an error.
- **throw**: sends (raises) the exception.
- **catch**: block that receives and handles the exception.
- Many `catch` blocks can follow one `try`; `catch(...)` catches all types.

- When an exception is thrown, objects made inside `try` are destroyed (**destructors are called**) before `catch` runs.
- **User-defined exception:** a class can be thrown. Needed when built-in types cannot describe the error (invalid age, low balance).

PROGRAM

Exception with constructor / destructor and a user-defined exception:

```
#include <iostream>
#include <string>
using namespace std;
class Test {
public:
    Test() { cout << "Constructor\n"; }
    ~Test() { cout << "Destructor\n"; }
};
class MyException {
public:
    string msg;
    MyException(string m) { msg = m; }
};
int main() {
    try {
        Test t;
        int a = 10, b = 0;
        if (b == 0) throw "Division by zero";
        cout << a / b;
    }
    catch (const char *m) {
        cout << "Caught: " << m << endl;
    }
    try {
        int age = 15;
        if (age < 18) throw MyException("Age must be 18 or more");
    }
    catch (MyException &e) {
        cout << "User-defined exception: " << e.msg << endl;
    }
    return 0;
}
```

CONCLUSION

Exception handling separates error handling from normal code and makes programs reliable.

Q3. What are the types of file access? Explain file operations with a program.

PYQ 2017 PYQ 2020 (unformatted I/O) PYQ 2021 PYQ 2022

INTRODUCTION

Data in variables is lost when the program ends. Files store data permanently.

DEFINITION

File handling is reading from and writing to disk files using stream classes from `<fstream>`.

POINTS

- Stream classes: **ifstream** (input), **ofstream** (output), **fstream** (both).
- Steps: create stream object, open file, read / write, `close()`.
- **Sequential access:** data read / written in order from the start (`<<`, `>>`, `get()`, `put()`, `getline()`).
- **Random access:** any position reached directly: `seekg(pos)` / `seekp(pos)` move get / put pointer; `tellg()` / `tellp()` give the position.
- Other functions: `eof()`, `fail()`, `good()`, `close()`.

PROGRAM

Write, read and random access:

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
int main() {
    // Write
    ofstream fout("student.txt");
    fout << "Aman 85" << endl;
    fout << "Riya 90" << endl;
    fout.close();

    // Read (sequential)
    ifstream fin("student.txt");
    string name; int marks;
    while (fin >> name >> marks)
        cout << name << " " << marks << endl;
    fin.close();

    // Random access
    ifstream f("student.txt");
    f.seekg(0, ios::end);           // go to end
    long size = f.tellg();          // position = file size
    cout << "File size = " << size << " bytes" << endl;
    f.seekg(0, ios::beg);          // back to start
    f.close();
    return 0;
}
```

CONCLUSION

File handling gives permanent storage and flexible access to data.

Q4. What are templates? Explain the need and how a normal function becomes a template function.

PYQ 2014 PYQ 2017 PYQ 2019 PYQ 2024 (generic classes) PYQ 2025

INTRODUCTION

A function like `max(int, int)` must be rewritten for `float`, `char` etc. Templates solve this.

DEFINITION

A **template** lets us write **one generic function or class** that works for any data type.

POINTS

- **Need:** removes code repetition, saves time, gives **type-safe generic programming**.
- **Normal to template:** write `template <class T>` before the function and replace the data type with `T`.
- Normal: `int minimum(int a, int b) { return (a < b) ? a : b; }`
- Template: `template <class T> T minimum(T a, T b) { return (a < b) ? a : b; }`
- **Class template** (generic class) works the same way for classes.

PROGRAM

Function template and class template:

```
#include <iostream>
using namespace std;
template <class T>
T minimum(T a, T b) { return (a < b) ? a : b; }

template <class T>
class Box {
    T v;
public:
    Box(T x) { v = x; }
    T get() { return v; }
};

int main() {
    cout << minimum(10, 20) << endl;        // int
    cout << minimum('a', 'z') << endl;      // char
    cout << minimum(3.5f, 2.1f) << endl;    // float
    Box<int> b1(5);
    Box<float> b2(2.5f);
    cout << b1.get() << " " << b2.get() << endl;
    return 0;
}
```

CONCLUSION

Templates give **code reuse for different data types**.

Q5. What is the role of manipulators in C++? Write down the different manipulators with examples.

PYQ 2021 PYQ 2022 PYQ 2025

INTRODUCTION

Displaying data clearly requires proper formatting. C++ provides a convenient tool called manipulators for this.

DEFINITION

Manipulators are special functions / objects used with << and >> operators to **format the input and output** of a program. They are used as if they are data items in an output or input expression.

POINTS: TYPES OF MANIPULATORS

- Header file: `<iomanip>` for parameterised and `<iostream>` for non-parameterised manipulators.
- **endl**: inserts a newline character and flushes the buffer. `cout << "Hi" << endl;`
- **setw(n)**: sets the **field width** for the NEXT output only. `cout << setw(10) << "Aman";`
- **setprecision(n)**: sets the number of **decimal places**. `cout << setprecision(2) << 3.14159; → 3.14`
- **setfill(c)**: sets the **fill character** for empty spaces in setw. `cout << setfill('*') << setw(8) << 42; → *****42`
- **left / right**: sets alignment of output inside the field. `cout << left << setw(10) << "Hi";`
- **hex / oct / dec**: prints integers in **hexadecimal / octal / decimal** base.
- **fixed**: prints floating numbers in **fixed notation** (not scientific). `cout << fixed << setprecision(2) << 3.14159;`
- **scientific**: prints in **scientific notation**. `cout << scientific << 3.14159; → 3.14e+00`
- **boolalpha**: prints bool as `true / false` instead of `1 / 0`.
- **flush**: flushes the output buffer without a newline.
- **ws**: skips leading whitespace on input.

PROGRAM

Demonstrating important manipulators together:

```
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
    // setw and setfill
    cout << setfill('.') << setw(20) << "" << endl;
    cout << setfill(' ') << setw(10) << left << "Name"
        << setw(5) << "Age" << endl;
    cout << setfill('.') << setw(20) << "" << endl;
    cout << setfill(' ') << setw(10) << left << "Aman"
        << setw(5) << 21 << endl;

    // setprecision + fixed
    cout << fixed << setprecision(2) << 3.14159 << endl; // 3.14

    // hex and oct
    cout << hex << 255 << endl; // ff
    cout << oct << 255 << endl; // 377
    cout << dec << 255 << endl; // 255

    // boolalpha
    cout << boolalpha << true << " " << false << endl; // true false
    return 0;
}
```

CONCLUSION

Manipulators make C++ I/O professional, readable and easy to format without changing the data itself.

Q6. Discuss the concepts of generalization and aggregation with examples. PYQ 2025**INTRODUCTION**

OOP models real-world relationships. Two fundamental relationships used in class design are generalization and aggregation.

DEFINITION

Generalization is an IS-A relationship (base to derived, implemented by inheritance). **Aggregation** is a HAS-A relationship (an object contains another object but both can exist independently).

POINTS: GENERALIZATION VS AGGREGATION

- **Generalization (IS-A):** a derived class IS-A type of the base class. Implemented by inheritance. Example: Dog IS-A Animal.
- **Aggregation (HAS-A, weak):** one class USES another. If the outer class is deleted, the inner object still exists. Example: Department HAS-A Teacher (Teacher exists even if Department is deleted). Implemented with a pointer.
- **Composition (HAS-A, strong):** the inner object CANNOT exist without the outer. Example: House HAS-A Room (Room does not exist without House). Implemented by value (not pointer).
- **Association:** a general relationship where two classes use each other (has-a but neither owns the other).
- Generalization promotes **code reuse** through inheritance.
- Aggregation promotes **code flexibility** (loose coupling).
- Generalization uses `: public Base` syntax; aggregation uses a **pointer or reference** data member.

```
Generalization:      Animal (base)
                    |
                    Dog (derived) -- Dog IS-A Animal

Aggregation (weak):  Department ----> Teacher
                    (Department HAS-A Teacher; Teacher exists without Department)

Composition (strong): House []----> Room
                    (Room does not exist without House)
```

EXAMPLE

Generalization via inheritance and aggregation via pointer:

```

#include <iostream>
using namespace std;

// Generalization (inheritance / IS-A)
class Animal {
public:
    void eat() { cout << "Animal eating\n"; }
};
class Dog : public Animal {    // Dog IS-A Animal
public:
    void bark() { cout << "Dog barking\n"; }
};

// Aggregation (HAS-A, weak) – Teacher exists independently
class Teacher {
public:
    string name;
    Teacher(string n) { name = n; }
};
class Department {
    Teacher* t;    // pointer = aggregation
public:
    Department(Teacher* tp) { t = tp; }
    void show() { cout << "Dept teacher: " << t->name << endl; }
};

int main() {
    Dog d; d.eat(); d.bark();           // generalization
    Teacher t("Rahul");
    Department dept(&t);
    dept.show();                       // aggregation
    return 0;
}

```

CONCLUSION

Generalization and aggregation are key OOP design concepts that model IS-A and HAS-A relationships for clean, reusable, and flexible code.

PYQ ALERT

Asked in: Virtual functions (2017, 2019, 2020, 2021, 2022, 2024); Exception handling (2014, 2017, 2018, 2020, 2022); File handling (2017, 2020, 2021, 2022); Templates (2014, 2017, 2019, 2024); Pointers to members (2014); Manipulators (2021, 2022, 2024); Dynamic binding (2020, 2022); Manipulators role + types (2025, 12-mark); Generalization and aggregation (2025, 12-mark); User-defined exception scenarios (2022, 2025); Function template program (2025); Exception handling definition (2025, 4-mark); Static data/functions (2025, 12-mark Unit 1)

Frequency: High

MOST REPEATED QUESTIONS

(All uploaded years)

#	Question / Topic	Asked in
1	Abstract class (what is it / explain / benefit)	2017, 2018, 2019, 2022, 2024
2	Types of inheritance	2014, 2017, 2018, 2021, 2024
3	Operator overloading (definition / program / restrictions)	2014, 2017, 2018, 2019, 2022 (short: 2020, 2021)
4	Constructor, types, characteristics	2014, 2017, 2018, 2019, 2020, 2022 (also 2021, 2024)
5	Friend function	2014, 2018, 2020, 2021, 2022, 2024
6	Virtual function / dynamic binding / virtual destructor	2017, 2019, 2020, 2021, 2022, 2024
7	OOP vs POP / features of OOP	2017, 2019, 2020, 2021, 2022, 2024
8	Exception handling (try, catch, throw)	2014, 2017, 2018, 2020, 2022
9	Copy constructor	2014, 2017, 2019, 2021
10	Templates (function / class / generic)	2014, 2017, 2019, 2024
11	File handling (sequential, random, operations)	2017, 2020, 2021, 2022
12	Inline function	2017, 2020, 2021
13	Function overloading	2014, 2019, 2020 (method overloading), 2022
14	Insertion / extraction operator with friend	2014, 2018
15	Virtual base class	2020, 2022
16	Polymorphism	2017, 2021, 2022
17	Access specifiers and visibility	2018, 2022
18	Manipulators	2021, 2022, 2024
19	Objects as function arguments	2020, 2021
20	Dynamic objects (new / delete)	2018, 2021
21	Data hiding / encapsulation	2022, 2024, 2025
22	Static data member and member functions	2019, 2020, 2025 (12-mark)
23	Manipulators (role + types)	2021, 2022, 2025 (12-mark)
24	User-defined exception	2022, 2025
25	Generalization and aggregation	2025
26	Token in C++	2025
27	Proxy classes	2025
28	Difference between C and C++ (code)	2020, 2025
29	Abstract class + Virtual base class (combined)	2025

TOP 20 VVI QUESTIONS

(12 Marks + 4 Marks)

PART A | 12 MARKS (Q1 to Q10)

1. Explain the object-oriented paradigm with its features. Differentiate between OOP and POP.

PYQ 2017 PYQ 2019 PYQ 2020 PYQ 2021 PYQ 2022 PYQ 2024

Full answer: Unit 1, Column 3, Q1 — page 7

2. What is a constructor? Explain its types and characteristics. Write a program for the copy constructor.

PYQ 2014 PYQ 2017 PYQ 2018 PYQ 2019 PYQ 2020 PYQ 2021 PYQ 2022 PYQ 2024

Full answer: Unit 1, Column 3, Q2 — page 7

3. What is a friend function? Why is it required? Write a program.

PYQ 2014 PYQ 2018 PYQ 2020 PYQ 2021 PYQ 2022 PYQ 2024

Full answer: Unit 1, Column 3, Q3 — page 8

4. What is operator overloading? Explain with restrictions. Write a C++ program to overload + (and -, *) for complex numbers.

PYQ 2014 PYQ 2017 PYQ 2018 PYQ 2019 PYQ 2022 PYQ 2025

Full answer: Unit 2, Column 3, Q1 — page 13

5. Explain the conversion of basic data type to class type and class type to basic type with example.

PYQ 2014 PYQ 2025

Full answer: Unit 2, Column 3, Q2 — page 13

6. What is inheritance? Describe various types of inheritance with example.

PYQ 2014 PYQ 2017 PYQ 2018 PYQ 2021 PYQ 2024 PYQ 2025

Full answer: Unit 3, Column 3, Q1 — page 18

7. Explain the visibility of base class members for the access specifiers private, protected and public while creating the derived class.

PYQ 2018 PYQ 2022

Full answer: Unit 3, Column 3, Q2 — page 18

8. Discuss virtual base class with an example. Also explain ambiguity in multiple inheritance and how to resolve it.

PYQ 2020 PYQ 2022 PYQ 2024 (ambiguity)

Full answer: Unit 3, Column 3, Q3 — page 19

9. What is a virtual function? Explain dynamic binding and abstract class with example.

PYQ 2017 PYQ 2019 PYQ 2020 PYQ 2021 PYQ 2022 PYQ 2024

Full answer: Unit 4, Column 3, Q1 — page 23

10. Define exception handling. Explain the use of try, catch and throw with example.

PYQ 2014 PYQ 2017 PYQ 2018 PYQ 2020 PYQ 2022

Full answer: Unit 4, Column 3, Q2 — page 23

11. What is the significance of static data members and static member functions in C++?

PYQ 2025

Full answer: Unit 1, Column 3, Q4 — page 9

12. What is the role of manipulators in C++? Write down the different manipulators with examples.

PYQ 2021 PYQ 2022 PYQ 2025

Full answer: Unit 4, Column 3, Q5 — page 26

13. Discuss the concepts of generalization and aggregation with examples. PYQ 2025

Full answer: Unit 4, Column 3, Q6 — page 27

PART B | 4 MARKS (Q11 to Q20): full answers**Q11. Characteristics of a constructor and destructor.** PYQ 2014 PYQ 2018 PYQ 2019 PYQ 2020**Ans.**

- **Constructor:** same name as class; no return type; called automatically; usually public; can be overloaded; can have default arguments; cannot be inherited or virtual.
- **Destructor:** name is `~ClassName`; no arguments, no return value; **only one per class**; called automatically when object goes out of scope or is deleted; cannot be overloaded.

Q12. Briefly explain the copy constructor. PYQ 2014 PYQ 2017 PYQ 2019 PYQ 2021**Ans.**

A **copy constructor** creates a new object as a **copy of an existing object** of the same class. Syntax: `ClassName(const ClassName &obj)`. It is called when an object is initialised with another object, passed by value or returned by value. Its argument must be a **reference**, otherwise it would call itself endlessly. Write your own when the class has **pointer members** (deep copy).

Q13. How to create and destroy objects dynamically? PYQ 2018 PYQ 2021**Ans.**

Use the **new** operator to create and **delete** to destroy. C++ has no garbage collector, so forgetting **delete** causes a **memory leak**.

```
Student *p = new Student(101);    // constructor runs
p->show();
delete p;                          // destructor runs, memory freed
int *a = new int[5]; delete[] a;   // array form
```

Q14. Differentiate between call by value and call by reference. PYQ 2019 PYQ 2024**Ans.**

Call by value	Call by reference
Copy of data is passed	Address / alias is passed
Original value does not change	Original value changes
Slower for big objects	Faster and efficient
<code>void f(int a)</code>	<code>void f(int &a)</code>

Q15. What do you mean by encapsulation and how is it implemented in C++? PYQ 2024**Ans.**

Encapsulation means binding data and the functions that use it into one unit. In C++ it is implemented using a **class**: data members are kept **private** and member functions are **public**, so data is reached only through those functions. This also gives data hiding and security.

Q16. What is polymorphism? Briefly explain its types. PYQ 2017 PYQ 2021 PYQ 2022**Ans.**

Polymorphism means "one name, many forms". **Compile-time (static):** function overloading and operator overloading. **Run-time (dynamic):** virtual functions with a base class pointer. Example: `add(int, int)` and `add(float, float)` share one name.

Q17. How can ambiguity in multiple inheritance be resolved? PYQ 2024**Ans.**

- Use the **scope resolution operator**, example `obj.A::show()`;
- **Override** the function in the derived class.
- Use a **virtual base class** when the same grand-base class comes through two paths.

Q18. Explain the virtual destructor. PYQ 2021 PYQ 2024**Ans.**

If a base pointer holds a derived object and the base destructor is **not virtual**, `delete p`; runs only the base destructor, which causes a **memory leak**. Declaring `virtual ~Base()` makes `delete p`; call the **derived destructor first, then the base**. Rule: any class with virtual functions should have a virtual destructor.

Q19. Discuss the need for exception handling with try, catch and throw. PYQ 2017**Ans.**

Without it, a run-time error such as divide by zero **stops the program abruptly**. With `try`, risky code is watched; `throw` raises the error; `catch` handles it and the program continues safely. It keeps **error code separate** from normal code and is cleaner than `if-else` checks everywhere.

Q20. Write a sample code to show the difference between C and C++. PYQ 2025**Ans.**

C is a **procedural** language (functions only). C++ is **object-oriented** (adds classes, objects, overloading, inheritance). The key addition in C++ is the ability to group data and functions inside a **class**.

```
// C style
#include <stdio.h>
void greet() { printf("Hello, C!
"); }
int main() { greet(); return 0; }

// C++ style
#include <iostream>
using namespace std;
class Greeter {
public:
    void greet() { cout << "Hello, C++!" << endl; }
};
int main() { Greeter g; g.greet(); return 0; }
```

Q21. Explain token in C++. PYQ 2025**Ans.**

A **token** is the **smallest unit** of a C++ program that the compiler recognises. Types: **Keywords** (`int`, `class`), **Identifiers** (variable/function names), **Constants / Literals** (`10`, `'A'`), **Operators** (`+`, `::`), **Punctuators** (`;`, `{`, `}`).

Q22. Define proxy classes. PYQ 2025**Ans.**

A **proxy class** provides a **controlled interface** to another (real) class. It hides the real class and can add features like lazy loading. C++ smart pointers (`unique_ptr`, `shared_ptr`) are proxy classes for raw pointers.

Q23. Discuss the role of manipulators in C++. PYQ 2021 PYQ 2022 PYQ 2025**Ans.**

Manipulators format I/O by modifying stream state. Key ones: `endl` (newline + flush), `setw(n)` (field width), `setprecision(n)` (decimal places), `setfill(c)` (fill character), `left/right` (alignment), `hex/oct/dec` (number base), `fixed` (fixed notation), `boolalpha` (bool as true/false). Header: `<iomanip>` (parameterised) and `<iostream>` (non-parameterised).

Q24. What is an abstract class? What is its benefit?

PYQ 2017

PYQ 2018

PYQ 2019

PYQ 2022

PYQ 2024

Ans.

An **abstract class** has at least one **pure virtual function** ($= 0$) and **cannot be instantiated**. Benefits: it gives a **common interface / base** for related classes, forces derived classes to implement required functions, and supports **run-time polymorphism**.

IMPORTANT C++ PROGRAMS

Program 1 — Constructor / Destructor / Copy Constructor (default values) PYQ 2014 PYQ 2018

```
#include <iostream>
using namespace std;
class Distance {
    int feet, inches;
public:
    Distance(int f = 0, int i = 0) {           // parameterized + default values
        feet = f; inches = i;
        cout << "Constructor called\n";
    }
    Distance(const Distance &d) {             // copy constructor
        feet = d.feet; inches = d.inches;
        cout << "Copy constructor called\n";
    }
    void show() { cout << feet << " ft " << inches << " in\n"; }
    ~Distance() { cout << "Destructor called\n"; }
};
int main() {
    Distance d1(5, 8);
    Distance d2 = d1;
    d1.show();
    d2.show();
    return 0;
}
```

Program 2 — Operator Overloading: +, -, *, <<, >> for Complex PYQ 2018 PYQ 2022

```
#include <iostream>
using namespace std;
class Complex {
    float re, im;
public:
    Complex(float r = 0, float i = 0) { re = r; im = i; }
    Complex operator+(Complex c) { return Complex(re + c.re, im + c.im); }
    Complex operator-(Complex c) { return Complex(re - c.re, im - c.im); }
    Complex operator*(Complex c) {
        return Complex(re * c.re - im * c.im, re * c.im + im * c.re);
    }
    friend ostream& operator<<(ostream &out, const Complex &c);
    friend istream& operator>>(istream &in, Complex &c);
};
ostream& operator<<(ostream &out, const Complex &c) {
    out << c.re << " + " << c.im << "i";
    return out;
}
istream& operator>>(istream &in, Complex &c) {
    in >> c.re >> c.im;
    return in;
}
int main() {
    Complex a(2, 3), b(4, 5);
    cout << "Sum = " << a + b << endl;
    cout << "Diff = " << a - b << endl;
    cout << "Prod = " << a * b << endl;
    return 0;
}
```

Program 2B — Prefix and Postfix Decrement using Friend Function PYQ 2019

```
#include <iostream>
using namespace std;
class Counter {
    int n;
public:
    Counter(int x = 0) { n = x; }
    friend Counter operator--(Counter &c);           // prefix  --c
    friend Counter operator--(Counter &c, int);      // postfix c--
    void show() { cout << n << endl; }
};
Counter operator--(Counter &c) { --c.n; return c; }
Counter operator--(Counter &c, int) { Counter t = c; c.n--; return t; }
int main() {
    Counter a(5), b(5);
    --a; a.show();      // 4
    b--; b.show();      // 4
    return 0;
}
```

Program 3 — Inheritance: Constructors and Destructors in Derived Class PYQ 2024

```
#include <iostream>
#include <string>
using namespace std;
class Person {
protected:
    string name;
public:
    Person(string n) { name = n; cout << "Person constructor\n"; }
    ~Person() { cout << "Person destructor\n"; }
};
class Student : public Person {
    int roll;
public:
    Student(string n, int r) : Person(n) { roll = r; cout << "Student constructor\n"; }
    void show() { cout << name << " " << roll << endl; }
    ~Student() { cout << "Student destructor\n"; }
};
int main() {
    Student s("Aman", 101);
    s.show();
    return 0;
}
// Order: Person ctor, Student ctor, show, Student dtor, Person dtor
```

Program 3B — Multiple Inheritance: Classes X, Y, Z (string concatenation) PYQ 2014

```

#include <iostream>
#include <cstring>
using namespace std;
class X {
protected:
    char a[20];
public:
    X(const char *s) { strcpy(a, s); }
    ~X() { cout << "X destroyed\n"; }
};
class Y {
protected:
    char b[20];
public:
    Y(const char *s) { strcpy(b, s); }
    ~Y() { cout << "Y destroyed\n"; }
};
class Z : public X, public Y {
    char c[40];
public:
    Z(const char *s1, const char *s2) : X(s1), Y(s2) {
        strcpy(c, a);
        strcat(c, b);
    }
    void show() { cout << a << endl << b << endl << c << endl; }
    ~Z() { cout << "Z destroyed\n"; }
};
int main() {
    Z obj("Hello", "World");
    obj.show();
    return 0;
}

```

Program 4 — Templates (function and class) PYQ 2014 PYQ 2019

```

#include <iostream>
using namespace std;
template <class T>
T minimum(T a, T b) { return (a < b) ? a : b; }

template <class T>
class Box {
    T v;
public:
    Box(T x) { v = x; }
    T get() { return v; }
};
int main() {
    cout << minimum(10, 20) << endl;           // int
    cout << minimum('a', 'z') << endl;         // char
    cout << minimum(3.5f, 2.1f) << endl;       // float
    Box<int> b1(5);
    Box<float> b2(2.5f);
    cout << b1.get() << " " << b2.get() << endl;
    return 0;
}

```

Program 5 — Exception Handling (constructor / destructor + user-defined exception)

PYQ 2014 PYQ 2022

```

#include <iostream>
#include <string>
using namespace std;
class Test {
public:
    Test() { cout << "Constructor\n"; }
    ~Test() { cout << "Destructor\n"; }
};
class MyException {
public:
    string msg;
    MyException(string m) { msg = m; }
};
int main() {
    try {
        Test t;
        int a = 10, b = 0;
        if (b == 0) throw "Division by zero";
        cout << a / b;
    }
    catch (const char *m) {
        cout << "Caught: " << m << endl;
    }
    try {
        int age = 15;
        if (age < 18) throw MyException("Age must be 18 or more");
    }
    catch (MyException &e) {
        cout << "User-defined exception: " << e.msg << endl;
    }
    return 0;
}

```

Program 6 — File Handling (sequential + random access)

PYQ 2017 PYQ 2021 PYQ 2022

```

#include <iostream>
#include <fstream>
#include <string>
using namespace std;
int main() {
    // Write
    ofstream fout("student.txt");
    fout << "Aman 85" << endl;
    fout << "Riya 90" << endl;
    fout.close();

    // Read (sequential)
    ifstream fin("student.txt");
    string name; int marks;
    while (fin >> name >> marks)
        cout << name << " " << marks << endl;
    fin.close();

    // Random access
    ifstream f("student.txt");
    f.seekg(0, ios::end);
    long size = f.tellg();           // go to end
    // position = file size
    cout << "File size = " << size << " bytes" << endl;
    f.seekg(0, ios::beg);           // back to start
    f.close();
    return 0;
}

```

Program 7 — Average Marks of 50 Students PYQ 2024

```
#include <iostream>
using namespace std;
int main() {
    const int N = 50;
    float marks[N], sum = 0;
    for (int i = 0; i < N; i++) {
        cout << "Enter marks of student " << i + 1 << ": ";
        cin >> marks[i];
        sum += marks[i];
    }
    cout << "Average marks = " << sum / N << endl;
    return 0;
}
// Assumption: marks are out of 100, stored as float
```

Program 8 — Static Data Member + Static Function (count objects) PYQ 2019 PYQ 2020 PYQ 2025

```
#include <iostream>
using namespace std;
class Student {
    int roll;
    static int count;
public:
    Student(int r) { roll = r; count++; }
    ~Student() { count--; }
    static void showCount() {
        cout << "Total Students: " << count << endl;
    }
};
int Student::count = 0;
int main() {
    Student::showCount(); // 0
    Student s1(1), s2(2);
    Student::showCount(); // 2
    {
        Student s3(3);
        Student::showCount(); // 3
    }
    Student::showCount(); // 2
    return 0;
}
```

Program 9 — User-Defined Exception (Bank Account / Insufficient Funds) PYQ 2022 PYQ 2025

```

#include <iostream>
#include <string>
using namespace std;
class InsufficientFunds {
public:
    double amount;
    string msg;
    InsufficientFunds(double a) {
        amount = a;
        msg = "Need Rs." + to_string((int)a) + " more!";
    }
};
class BankAccount {
    double balance;
public:
    BankAccount(double b) { balance = b; }
    void withdraw(double a) {
        if (a > balance) throw InsufficientFunds(a - balance);
        balance -= a;
        cout << "Withdrawn Rs." << a
              << " Balance: Rs." << balance << endl;
    }
};
int main() {
    try {
        BankAccount acc(1000);
        acc.withdraw(500);    // OK
        acc.withdraw(700);    // throws
    }
    catch (InsufficientFunds &e) {
        cout << "Error: " << e.msg << endl;
    }
    return 0;
}

```

Program 10 — Function Template: Product of Two Values (int, float, char) PYQ 2025

```

#include <iostream>
using namespace std;
template <class T>
T product(T a, T b) { return a * b; }
int main() {
    cout << product(4, 5) << endl;    // 20    (int)
    cout << product(2.5f, 3.0f) << endl;    // 7.5    (float)
    // Note: char multiplication gives ASCII product as int
    cout << (int)product('A', 'A') << endl;    // 130
    return 0;
}

```

LAST MINUTE CHEAT SHEET

One-liner Definitions

- **Class** = blueprint; **Object** = instance.
- **Encapsulation** = wrap data + functions; **Abstraction** = hide complexity; **Data hiding** = private data.
- **Inheritance** = reuse; **Polymorphism** = one name, many forms.
- **Constructor** = same name as class, no return type; **Destructor** = `~Name`.
- **Friend** = non-member with access to private data.
- **Static** = one shared copy; **this** = address of the calling object.
- **Virtual function** = run-time choice; **Pure virtual** = `= 0`; **Abstract class** = has a pure virtual function.
- **Template** = generic code for any type.
- **Exception** = run-time error handled with try / catch / throw.
- **Stream** = flow of data; `ifstream` read, `ofstream` write, `fstream` both.

Common Mistakes

- Writing a **return type** for a constructor or destructor.
- Forgetting the **semicolon after the class closing brace** `};`.
- Defining a friend function with `A::` or the word `friend` outside the class.
- Using a **base class pointer to call derived-only functions**.
- Forgetting `virtual` in the base class, so the base function runs.
- Trying to create an **object of an abstract class**.
- Forgetting to **close the file** or to check that it opened.
- Writing a `catch` with the wrong type, so the exception is not caught.
- Creating an object with no arguments when only a parameterized constructor exists (compile error).
- Passing the copy constructor argument **by value** instead of by reference.

Memory Tricks

- **OOP features = "EAPI"**: Encapsulation, Abstraction, Polymorphism, Inheritance.
- **Access specifiers = "PPP"**: Private (class default), Protected, Public.
- **Constructor types = "DPC"**: Default, Parameterized, Copy.
- **Inheritance types = "SMMHH"**: Single, Multiple, Multilevel, Hierarchical, Hybrid.
- **Cannot overload**: Dot `.`, Scope `::`, Ternary `?:`, `sizeof`, Star-dot `.*`.
- **Exception = "TCT"**: Try, Catch, Throw.
- **Order rule**: Constructors go **Base to Derived**; Destructors go **Derived to Base**.
- **Binary operator arguments**: member = **1**, friend = **2** (friend has no `this`).
- **Private base members are never directly usable in derived; protected can be used in derived but not outside.**

Important Syntax

```

class A { private: int x; public: A(int a){x=a;} ~A(){} }; // class
void A::show() { } // outside definition
Complex operator+(Complex c); // operator function
friend ostream& operator<<(ostream&, const A&); // insertion
class B : public A { }; // single inheritance
class C : public A, public B { }; // multiple
class D : virtual public A { }; // virtual base
virtual void f(); virtual void g() = 0; // virtual / pure virtual
template <class T> T f(T a, T b) { } // template
try { throw 10; } catch (int e) { } // exception
ofstream fout("a.txt"); ifstream fin("a.txt"); // files
Student *p = new Student(1); delete p; // dynamic object

```

Source note: the 2017 first page is partly cut in the scan (Q2 marks) and the 2014 MCQs (f) and (g) have unclear options, so their answers are Not verified from uploaded material. Papers for 2015, 2016, 2023 and 2025 were not uploaded. MCQs marked Practice are made from the syllabus, not from a PYQ. No question here is guaranteed to come.